DP-OPS-0020: Phase Two Doctrine — The Economy of Truth, Context Hygiene, and Uniform Style

Executive Summary: The Transition to High-Density Governance

The evolution of the Stela platform has reached a critical inflection point. Phase One, defined by the establishment of the *Stela Filing Doctrine* (DP-OPS-0001 through DP-OPS-0018), successfully arrested the entropy of the codebase by imposing rigid structural governance. This phase introduced the canonical triad: the *Constitution* (TRUTH.md), the *History Ledger* (SoP.md), and the *Active Contract* (TASK.md). These artifacts served to partition authority between the Human Operator, the AI Integrator, and the AI Contractor, establishing a functional, albeit verbose, collaborative environment.[1] However, the very success of Phase One has precipitated a secondary crisis: **Context Saturation**. As the governance layer has matured, it has expanded in lexical volume, consuming an increasingly disproportionate share of the finite context window available to autonomous agents.

We observe that the current system suffers from "Narrative Debt"—the accumulation of explanatory prose, philosophical justifications, and redundant safety rails that, while human-readable, constitute "noise" for a Large Language Model (LLM). This noise dilutes the "Attention Budget" of the agent, leading to probabilistic drift, slower inference times, and increased operational costs.[2] The "Context Window Crisis" is not merely a technical limitation but a governance failure; specifically, a failure to optimize the information density of the control plane.

Phase Two (DP-OPS-0020) proposes a radical restructuring of the governance layer, transitioning from a philosophy of *comprehensive explanation* to one of *axiomatic density*. This new doctrine is predicated on three pillars:

1. **Economy of Truth:** A reductionist approach to documentation that enforces maximum information density. We move from "instructional narrative" to "executable axioms," systematically stripping tokens that do not alter the probability distribution of the agent's desired output.
2. **Context Hygiene:** The shift from manual state management to automated, algorithmic pruning. The SoP.md ledger and storage/ artifacts will be managed as a "Sliding Window," ensuring that the historical context remains fresh, relevant, and bounded by strict token limits.[2]
3. **Uniform Style:** The enforcement of **Lexical Isomorphism**. By standardizing the syntactic structure of all markdown artifacts via automated linting, we reduce the cognitive load

required for the agent to parse the repository structure, thereby increasing the reliability of tool use and file navigation.[5]

This report provides the exhaustive theoretical framework, forensic audit of current inefficiencies, and precise technical specifications required to execute Phase Two. It integrates lessons from advanced context engineering [3], minimalist technical writing [7], and automated system maintenance [9] to propose a self-sustaining, autopoietic governance model.

Part I: The Crisis of Saturation (Theoretical Framework)

To justify the radical interventions proposed in DP-OPS-0020, we must first establish the theoretical basis for "Context Engineering" as a distinct discipline within AI Ops. The repository is not merely a file system; it is the "cognitive environment" of the agent. The quality of the agent's output is strictly deterministic based on the signal-to-noise ratio of this environment.

1.1 The Token-Utility Curve and Context Economics

In an agentic system, "Truth" is a resource quantified in tokens. Every word in the canonical context (CONTEXT.md) competes for space with the actual payload (the code in projects/). The **Token-Utility Curve** describes the relationship between the volume of instructions and the fidelity of adherence. Contrary to human intuition, adding more instructions does not linearly increase adherence. Instead, it follows a bell curve:

- **Insufficient Context:** The agent hallucinates due to a lack of constraints.
- **Optimal Context (The "Goldilocks Zone"):** The agent has precise, non-conflicting constraints and maximizes performance.[2]
- **Saturated Context:** The agent suffers from "Middle Drift," where instructions placed in the middle of a massive context window are attenuated or ignored.[10]

Current analysis of the Stela dump [1] indicates that we have entered the "Saturated Context" zone. The AGENTS.md file, for instance, dedicates significant token volume to explaining *why* rules exist (e.g., "This ensures rhythmic precision for technical auditing"). While pedagogically sound for humans, this is operational waste for an LLM. An AI model does not require persuasion; it requires constraint. The "Economy of Truth" doctrine asserts that **every token in the canonical context must purchase a commensurate amount of agentic behavior.** If a sentence does not effectively prune the search space of possible actions, it is noise.[3]

1.2 The Signal-to-Noise Ratio in Governance

Noise in a governance system is defined as any information that does not contribute to the

"Definition of Done." High-density documentation improves performance by reducing the "distraction factor." Research into retrieval-augmented generation (RAG) and long-context models demonstrates that including irrelevant or redundant documents significantly degrades performance.[3]

In Stela's current state, noise manifests in three forms:

1. **Redundancy:** The definition of "Drift" appears in AGENTS.md, TRUTH.md, and docs/GOVERNANCE.md.[1] This repetition forces the model to attend to three separate locations for a single concept, diluting the attention mechanism.
2. **Narrative:** Instructions are wrapped in conversational prose ("We value structure over memory"). This requires the model to parse the sentence structure to extract the underlying boolean logic (Structure > Memory).
3. **Obsolescence:** The SoP.md ledger grows linearly. Entries from months ago describing minor formatting fixes (DP-OPS-0009 [1]) remain in the active context, forcing the model to process historical data that has zero relevance to the current task.

1.3 The "Attention Budget" and Middle Drift

Transformer architectures utilize an attention mechanism to weigh the relevance of input tokens. However, this mechanism is not infinite. Empirical studies on "Lost in the Middle" phenomena [10] suggest that LLMs prioritize information at the beginning (System Prompt/Constitution) and the end (Current Task) of the context window. Information buried in the middle—such as a bloated SoP.md or a verbose MANUAL.md—is statistically less likely to be recalled accurately.

Phase Two addresses this by physically reducing the size of the "Middle." By compressing AGENTS.md (the beginning) and pruning SoP.md (the middle), we physically bring the Constitution and the Active Task closer together in the token stream, increasing the "Attention Voltage" between the rule and the execution.

Part II: Economy of Truth (The Axiomatic Shift)

The first pillar of Phase Two is the **Economy of Truth**. This involves a rigorous refactoring of all "Canon" documents to strip away narrative ("flavor text") and leave only the "Axioms" (executable logic). We treat English prose as a legacy programming language that must be compiled down to a more efficient intermediate representation.

2.1 Forensic Audit of Current Canon

A line-by-line analysis of the provided repository dump [1] reveals specific opportunities for compression. The following table contrasts the current "Phase One" text with the proposed

"Phase Two" axiomatic standard.

Table 1: Comparative Analysis of Governance Artifacts

File	Current Text (Phase One)	Proposed Axiom (Phase Two)	Reduction	Rationale
AGENTS.md	"Refrain from using contractions (e.g., 'don't,' 'can't'). This ensures rhythmic precision for technical auditing."	"No contractions."	**-85%**	The "why" (rhythmic precision) is irrelevant to execution. The list of examples is unnecessary for a literate model.
AGENTS.md	"Specify exact deviations from established protocols (e.g., 'execution path deviated by 14.7%') rather than using subjective descriptions."	"Report quantitative deviations only. No subjectivity."	**-60%**	Removes verbose phrasing ("established protocols") in favor of direct imperative ("Report").
AGENTS.md	"Actively monitor for structural anomalies. Logic or files not aligned with TRUTH.md	"Drift: TRUTH.md violation = System Failure."	**-70%**	"Actively monitor" is implied by the role. The definition of Drift is reduced to a boolean

	coordinates represent a system failure."			equality.
TRUTH.md	"The 4-Line Heuristic: 1. Does it RUN the system? -> ops/... 2. Does it EXPLAIN the system? -> docs/..."	"**Filing:** ops/=Run, docs/=Explain, projects/=Work, storage/=Trash."	**-65%**	Converts a 4-line list into a single-line key-value mapping.
TASK.md	"DISPOSABLE ARTIFACTS (chat logs, etc.) MUST NOT BE REFERENCED OR INCLUDED."	"Exclude disposable artifacts (chat/*, logs/*)."	**-55%**	Replaces shouting caps with standard constraints. Removes ambiguous "etc."
SoP.md	"Purpose: Reduce DP drafting deadlocks by allowing provisional branch and Base HEAD proposals while preserving worker git authority."	"Shipped: Provisional branch/HEAD proposal protocol."	**-75%**	The ledger needs to record *what* shipped, not the strategic intent (which belongs in the DP archive).

2.2 The "No-Narrative" Rule

Phase Two explicitly forbids "Narrative Technical Writing" [12] within the ops/ and docs/canon/ directories. While narrative techniques (analogies, story-framing) are powerful for human education, they are inefficient for AI instruction following. The AI does not need to be

"engaged" or "entertained"; it needs to be "programmed".[13]

New Invariant for TRUTH.md:

> *Documentation residing in ops/ and TRUTH.md must be Axiomatic. Use Subject-Verb-Object structures. Avoid adjectives. Avoid justification clauses ("because", "so that").*

This aligns with the "Minimalist" documentation philosophy [7], which advocates for stripping all non-essential information to support the user's immediate action. For an AI agent, the "immediate action" is the generation of the next token. Any token that does not constrain the search space for the next token is waste.

2.3 Redundant Definitions and Canonical Pointers

A major source of bloat is the repetition of core concepts. The concept of "Drift" is defined in AGENTS.md, TRUTH.md, and docs/GOVERNANCE.md. This violates the DRY (Don't Repeat Yourself) principle.

Remediation Strategy:

1. **Define Once:** TRUTH.md becomes the sole definition of "Drift" and "Structure."
2. **Point Everywhere:** All other documents must reference TRUTH.md.
    - *Current AGENTS.md:* "Logic or files not aligned with TRUTH.md coordinates represent a system failure."
    - *New AGENTS.md:* "Enforce TRUTH.md invariants."

This reduces the risk of "Definition Drift," where the definition of a rule in AGENTS.md slightly diverges from TRUTH.md over time, causing agent confusion.

2.4 The "Explanation" Trap

In docs/library/skills/S-LEARN-06.md (Advanced Git Forensics), the text warns of "The Trap: 'I remember where the bug is.' (Human memory is faulty)".[1] This is a "Human-Centric" instruction. An AI agent has perfect memory within its context window and zero memory outside of it. It does not suffer from "faulty memory" in the biological sense.

Phase Two Standard:

Instructions must be "Agent-Centric."

- *Bad:* "Don't rely on memory because humans forget."
- *Good:* "Verify state via git status. Do not assume state."

This shift removes the "anthropomorphic noise" [14]—instructions that treat the AI as a human being with human failings (fatigue, forgetfulness, ego) rather than a stochastic processing

unit with specific architectural constraints (context window, hallucination).

Part III: Context Hygiene (The Automated Ledger)

The second pillar of DP-OPS-0020 is **Context Hygiene**. In Phase One, hygiene was a manual responsibility of the Operator. In Phase Two, hygiene becomes an automated function of the System. This transition is critical because "Context Rot"—the accumulation of stale, irrelevant, or duplicate information—is the primary cause of agent degradation over long task chains.[3]

3.1 The Sliding Window Doctrine

The SoP.md file serves as the "History Ledger." However, an infinite append-only ledger is technically unsustainable in a context-limited environment. We formally establish the **Sliding Window Doctrine**:

- **Active Context:** The system requires high-fidelity knowledge of the *recent past* (e.g., the last 30 state changes) to maintain continuity.
- **Archived Context:** Knowledge older than the window is moved to "Cold Storage" (storage/archives/). It is accessible if explicitly requested but does not occupy the "Hot Context" loaded by ops/bin/open.

This mirrors the memory hierarchy in computer architecture (L1 Cache vs. RAM vs. Disk). SoP.md is the L1 Cache. It must be small, fast, and high-signal.

3.2 Automated Pruning: ops/bin/prune

We cannot rely on human discipline to maintain the Sliding Window. We must implement a binary, ops/bin/prune, that enforces these limits programmatically. This script will run as part of the closeout routine for every Dispatch Packet.

3.2.1 Technical Specification

The prune script will execute the following logic [4]:

1. **Target Selection:** Identify SoP.md.
2. **Count Headers:** Parse the file to count the number of ## YYYY-MM-DD entries.
3. **Threshold Check:** If count > MAX_ENTRIES (set to 30), trigger pruning.
4. **Cut Point Identification:** Identify the line number of the 31st header using grep and sed.
5. **Archival:**
    - Extract lines from the Cut Point to the End of File.
    - Append these lines to storage/archives/root/SoP-archive-YYYY-MM.md.
6. **Truncation:** Delete the extracted lines from SoP.md in place.
7. **Verification:** Ensure SoP.md is now within limits and the archive file has grown by the corresponding byte count.

Bash Implementation Logic:

Bash

```
#!/bin/bash
# ops/bin/prune - Context Hygiene Enforcement

SOP_FILE="SoP.md"
ARCHIVE_DIR="storage/archives/root"
MAX_ENTRIES=30

# Ensure archive directory exists
mkdir -p "$ARCHIVE_DIR"

# 1. Count current entries (ATX headers level 2 with date)
ENTRY_COUNT=$(grep -c "^## [0-9]\{4\}-[0-9]\{2\}-[0-9]\{2\}" "$SOP_FILE")

if; then
  echo "Pruning SoP.md: $ENTRY_COUNT entries found (Max: $MAX_ENTRIES)."

  # 2. Identify the line number of the 31st entry (The Cut Point)
  CUT_LINE=$(grep -n "^## [0-9]\{4\}-" "$SOP_FILE" | head -n $((MAX_ENTRIES + 1)) | tail -n 1 |
cut -d: -f1)

  # 3. Define Archive Target (Monthly rotation)
  ARCHIVE_FILE="$ARCHIVE_DIR/SoP-archive-$(date +%Y-%m).md"

  # 4. Append Old Entries to Archive
  echo "" >> "$ARCHIVE_FILE"
  tail -n +$CUT_LINE "$SOP_FILE" >> "$ARCHIVE_FILE"

  # 5. Truncate Active File (Delete from Cut Point to End)
  sed -i "${CUT_LINE},\$d" "$SOP_FILE"

  echo "Success: Moved $((ENTRY_COUNT - MAX_ENTRIES)) entries to $ARCHIVE_FILE."
else
  echo "SoP.md is within hygiene limits."
fi
```

Note: This script uses standard GNU tools (grep, sed, tail) available in the environment.[17]

3.3 Storage Hygiene and Artifact Lifecycle

The storage/ directory accumulates "Disposable Artifacts" such as OPEN prompts and OPEN-PORCELAIN diffs.[1] These are necessary for the *current* task but become noise immediately upon completion.

- **Rule:** storage/handoff/ is a temporary buffer.
- **Automation:** The ops/bin/prune script will also scan storage/handoff/.
- **Policy:** Delete any file older than 7 days matching the pattern OPEN-* or OPEN-PORCELAIN-*.[9]
- **Exception:** Files referenced in the currently active TASK.md must be locked and preserved. This requires parsing TASK.md to extract the DP-ID and excluding matching files from the deletion list.

3.4 Context Linter Upgrades

The current tools/context_lint.sh checks only for the *presence* of files.[1] Phase Two upgrades this to check for *Compliance*.

- **Size Guard:** The linter will fail if SoP.md exceeds 500 lines or 15KB. This forces the Operator (or the Agent) to run prune before work can proceed.[19]
- **Purity Check:** The linter will verify that ops/lib/manifests/CONTEXT.md does not contain forbidden paths (e.g., projects/ or storage/), strictly enforcing the separation between Platform Context and Project Payload.

Part IV: Uniform Style (Lexical Isomorphism)

The third pillar, **Uniform Style**, addresses the cognitive load associated with parsing inconsistent formatting. LLMs are sensitive to tokenization patterns. Inconsistent formatting (e.g., mixing ## Header and ##Header) creates "ragged" attention maps.[13] By enforcing **Lexical Isomorphism**—where identical semantic structures always have identical syntactic representation—we optimize the repository for machine readability.

4.1 The Necessity of Isomorphism

In a human team, style guides are often aesthetic. In an AI-augmented team, they are functional.

- **Headers:** ATX style (## Header) is preferred over Setext (underlined) because it uses fewer tokens and is easier to parse via regex.[6]
- **Lists:** Hyphens (-) are preferred over asterisks (*) for unordered lists to avoid confusion with bold/italic syntax **bold**.

- **Spacing:** Vertical spacing (blank lines between blocks) is critical for the model to identify section boundaries.[21]

4.2 Automated Style Enforcement: tools/lint_style.sh

We will replace the ad-hoc spelling checks of tools/lint_truth.sh [1] with a robust, industry-standard linter configuration using markdownlint.[5]

Proposed Configuration (.markdownlint.json):

JSON

```
{
  "default": true,
  "MD001": true,  // Header levels must increment by one
  "MD003": { "style": "atx" }, // Enforce ATX headers (# Header)
  "MD004": { "style": "dash" }, // Enforce hyphens for lists
  "MD007": { "indent": 2 }, // Enforce 2-space indentation
  "MD009": true,  // No trailing spaces
  "MD013": false, // Disable line length check (handled by soft wrap)
  "MD018": true,  // Space after hash on ATX header
  "MD022": true,  // Headers must be surrounded by blank lines
  "MD031": true,  // Fenced code blocks must be surrounded by blank lines
  "MD041": false  // Allow file to start without H1 (for snippets)
}
```

The wrapper script tools/lint_style.sh will execute this linter against all docs/, ops/, and root markdown files. This ensures that every file in the repository "looks" the same to the agent, reducing the probability of parsing errors.

4.3 The Uniform Header Doctrine

A review of the dump shows chaotic numbering schemes: 1., 1), II), and A) are used interchangeably in TASK.md.[1] This confuses the model's ability to track hierarchy depth.

Phase Two Standard:

- **Top Level:** # Title
- **Section Level:** ## N. Section Title (Decimal numbering).
- **Sub-Section:** ### N.N. Sub-Section.
- **Prohibited:** Roman numerals (II), parenthetical numbering 1), and non-numbered

headers for sequential steps.

This decimal standard (1.1, 1.2) helps the agent maintain "positional awareness" within deep documents.[23]

4.4 The "Code Fence" Rule (Refining DP-OPS-0009)

DP-OPS-0009 introduced the requirement to fence DP outputs.[1] Phase Two universalizes this.

- **Constraint:** Agents must *never* emit raw text that could be interpreted as Markdown structure (e.g., diffs, JSON, YAML) unless it is enclosed in triple backticks.
- **Reasoning:** This prevents the chat interface from rendering the artifacts, keeping them as "raw data" that the Operator can copy-paste without formatting errors. It also clearly delimits "Agent Voice" from "System Artifacts."

Part V: Operationalizing DP-OPS-0020

The implementation of Phase Two requires a structured rollout to prevent system shock. We will execute this via a dedicated Dispatch Packet.

5.1 DP-OPS-0020 Specification

Title: Phase Two Doctrine Implementation

Objective: Transition Stela to High-Density Governance via Pruning and Standardization.

Target Files (Allowlist): TRUTH.md, AGENTS.md, SoP.md, TASK.md, ops/bin/prune, tools/lint_style.sh, .markdownlint.json, .github/workflows/style_policing.yml, docs/GOVERNANCE.md.

5.2 Implementation Roadmap

Phase 2.1: The Great Compression (Economy)

1. **Refactor TRUTH.md:** Consolidate the "Hard Constraints" and "Invariants" into a single section. Remove the "Architecture Layers" narrative.
2. **Refactor AGENTS.md:** Rewrite Section 2 using the Axiomatic Standard. Remove the "Staffing Protocol" preamble. Convert the "Drafting Proposal Protocol" into a simple checklist.
3. **Audit:** Verify that the file size of AGENTS.md has decreased by at least 40%.

Phase 2.2: The Pruning Infrastructure (Hygiene)

1. **Develop ops/bin/prune:** Implement the logic defined in Section 3.2.
2. **Dry Run:** Execute ops/bin/prune locally. Verify that it correctly identifies the 31st entry in

SoP.md and generates the correct archive file.
3. **Baseline Reset:** Manually commit the first pruning operation to reset the SoP.md to the most recent 30 entries.
4. **Task Integration:** Update TASK.md to include ops/bin/prune in the "Closeout" checklist.

Phase 2.3: Style Enforcement (Uniformity)

1. **Configure Linter:** Commit .markdownlint.json and tools/lint_style.sh.
2. **CI Integration:** Create .github/workflows/style_policing.yml to block PRs that violate the style guide.
3. **Remediation:** Run the linter on the entire repository and fix existing violations (e.g., converting 1) to 1. in TASK.md).

5.3 Risk Management

- **Risk: Over-Pruning.** The prune script might inadvertently delete the current task's logs if the regex is too broad.
  - *Mitigation:* The script must use git status to identify "untracked" files and explicitly exclude them from deletion if they are younger than 24 hours.[18]
- **Risk: Context Anemia.** The "Economy" rewrite might remove nuance that was actually functional.
  - *Mitigation:* We will institute a "Shadow Mode" for the first 3 DPs, where the Operator closely monitors the agent for "Instruction Slippage." If the agent fails to follow a compressed rule, we will re-inflate that specific rule with *targeted* keywords, not full narrative.
- **Risk: Linter Fatigue.** A strict linter can block valid work due to minor formatting issues.
  - *Mitigation:* We will disable MD013 (Line Length) [6] to prevent the linter from fighting with the agent's natural output wrapping.

Part VI: Future Implications (Autopoiesis)

The ultimate goal of Phase Two is **Autopoiesis**—the creation of a self-maintaining system.[24] By automating the hygiene (Pruning) and policing the structure (Linting), we remove the Operator from the maintenance loop.

6.1 Second-Order Effects on Agent Behavior

We anticipate specific shifts in agent behavior post-implementation:

1. **Lower Latency:** Loading a compressed AGENTS.md and a pruned SoP.md will reduce the "Time to First Token" and lower API costs.[3]
2. **Higher Fidelity:** By removing the "noise" of justification, the "signal" of the constraint becomes stronger. The agent is less likely to "argue" or "explain" its actions and more likely to simply execute them.

3. **Transactional Culture:** The system will shift from a "Conversational" interface to a "Transactional" interface. The agent will act less like a chatty assistant and more like a deterministic kernel extension.

6.2 The Metric of Success

We will measure the success of DP-OPS-0020 by the **Equilibrium Stability Score**:

- *Target:* Zero manual edits to SoP.md for hygiene purposes.
- *Target:* <5% of PRs failing the Style Gate after Week 1.
- *Target:* 100% of new artifacts passing the "Axiomatic Test" (no narrative prose).

Phase Two represents the maturity of the Stela platform. It is the transition from "Building the System" to "Optimizing the Machine." By treating text as code, we ensure that our governance scales with our ambition.

End of Report.

Works cited

1. dump-full-work-proposal-protocol-2026-02-03-64cfae6ed.txt
2. Effective context engineering for AI agents - Anthropic, accessed February 4, 2026, https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents
3. What is Context Engineering? Architecting Reliable AI | Elastic, accessed February 4, 2026, https://www.elastic.co/what-is/context-engineering
4. Write shell script to move oldest file in the directory to the new directory.? - Stack Overflow, accessed February 4, 2026, https://stackoverflow.com/questions/58092619/write-shell-script-to-move-oldest-file-in-the-directory-to-the-new-directory
5. igorshubovych/markdownlint-cli: MarkdownLint Command Line Interface - GitHub, accessed February 4, 2026, https://github.com/igorshubovych/markdownlint-cli
6. markdownlint/docs/RULES.md at main - GitHub, accessed February 4, 2026, https://github.com/markdownlint/markdownlint/blob/main/docs/RULES.md
7. Exploring Minimalist Writing in Technical Documents, accessed February 4, 2026, https://www.sheaws.com/exploring-minimalist-writing-in-technical-documents/
8. Less is more: Minimalism in documentation - KnowledgeOwl, accessed February 4, 2026, https://www.knowledgeowl.com/blog/posts/minimalism-documentation
9. Automating File Cleanup: Removing Old Files with a Bash Script - Reliable Penguin - Blog, accessed February 4, 2026, https://blogs.reliablepenguin.com/2025/02/03/automating-file-cleanup-removing-old-files-with-a-bash-script
10. Long-Context LLMs Meet RAG: Overcoming Challenges for Long Inputs in RAG - arXiv, accessed February 4, 2026, https://arxiv.org/html/2410.05983v1

11. Optimizing LLM Accuracy | OpenAI API, accessed February 4, 2026, https://platform.openai.com/docs/guides/optimizing-llm-accuracy
12. The Different Flavours of Narrative Technical Writing - The Tech Writers Stack, accessed February 4, 2026, https://techwriters.substack.com/p/narrative-technical-writing-and-storytelling
13. Effective Prompts for AI: The Essentials - MIT Sloan Teaching & Learning Technologies, accessed February 4, 2026, https://mitsloanedtech.mit.edu/ai/basics/effective-prompts/
14. Docs for AI agents : r/technicalwriting - Reddit, accessed February 4, 2026, https://www.reddit.com/r/technicalwriting/comments/1ll6gzl/docs_for_ai_agents/
15. Shell script for moving oldest files? - Unix & Linux Stack Exchange, accessed February 4, 2026, https://unix.stackexchange.com/questions/22674/shell-script-for-moving-oldest-files
16. File rotation with bash script < Vielhuber David, accessed February 4, 2026, https://vielhuber.de/en/blog/file-rotation-with-bash-script/
17. Moving files/directories older than 7 days - Stack Overflow, accessed February 4, 2026, https://stackoverflow.com/questions/5963859/moving-files-directories-older-than-7-days
18. Script for moving files that are older than 5 days - Ask Ubuntu, accessed February 4, 2026, https://askubuntu.com/questions/698409/script-for-moving-files-that-are-older-than-5-days
19. get minimum and maximum line count from files within a directory, accessed February 4, 2026, https://unix.stackexchange.com/questions/611910/get-minimum-and-maximum-line-count-from-files-within-a-directory
20. Basic Syntax - Markdown Guide, accessed February 4, 2026, https://www.markdownguide.org/basic-syntax/
21. Markdown style guide | Gruntwork Docs, accessed February 4, 2026, https://docs.gruntwork.io/guides/style/markdown-style-guide/
22. markdownlint configuration in MegaLinter, accessed February 4, 2026, https://megalinter.io/8/descriptors/markdown_markdownlint/
23. Markdown style guide | styleguide - Google, accessed February 4, 2026, https://google.github.io/styleguide/docguide/style.html
24. Chapter 10 Introduction to Axiomatic Design, accessed February 4, 2026, https://ocw.mit.edu/courses/2-800-tribology-fall-2004/7956138e737f6cb1e9fb744fcc6c48ef_ch10_axiomatic.pdf